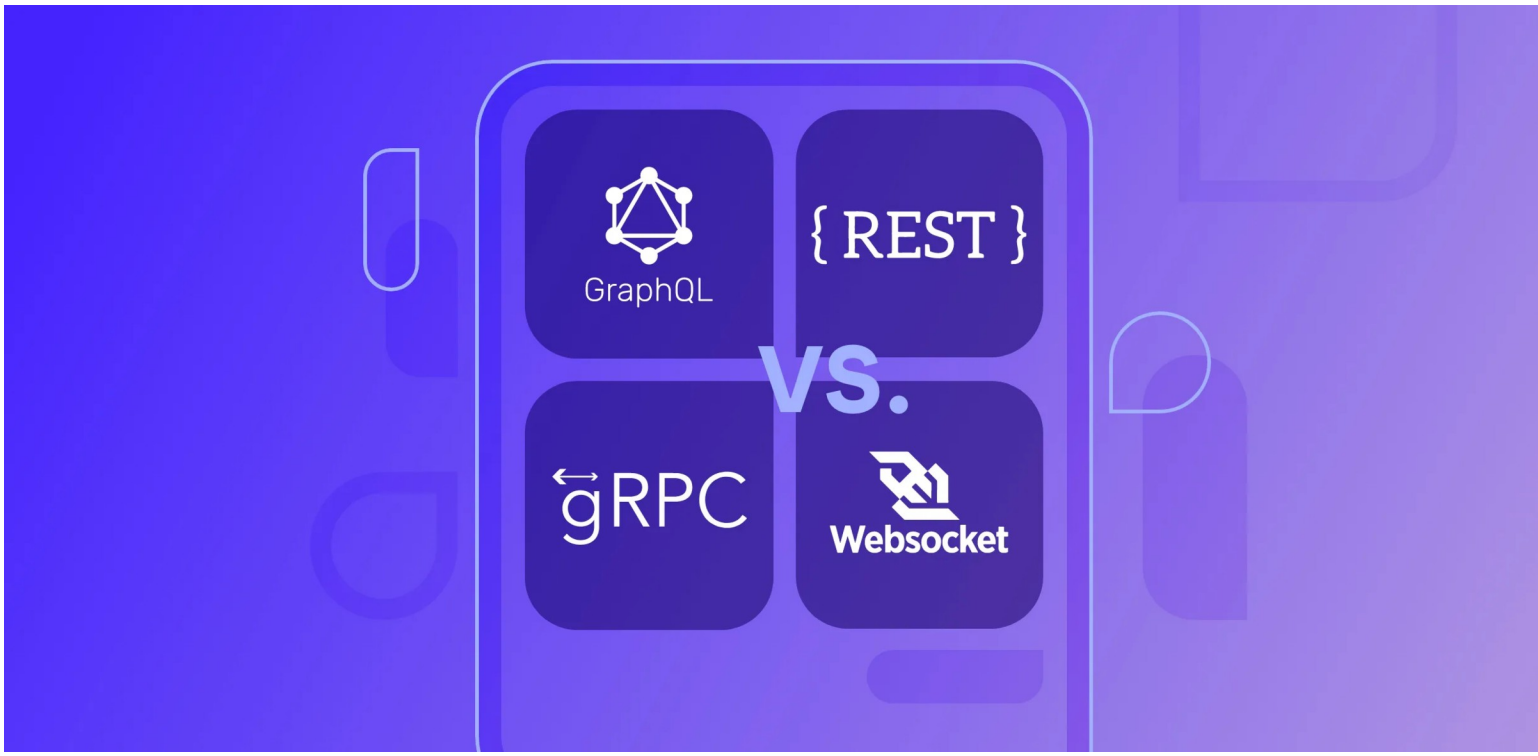
Informe Tecnico: REST, GraphQL y WebSocket.



1. Introducción.

El desarrollo moderno exige que los sistemas software intercambien datos de forma óptima, escalable y segura. Históricamente, las decisiones de arquitectura se tomaban en base a las herramientas existentes. Pero a día de hoy con las herramientas distribuidas, la elección elección de la arquitectura, puede impactar en el entorno de la red.

En este informe, veremos las tres formas de comunicación cliente-servidor: REST, GraphQL y WebSocket.

2. Desarrollo de la tecnología.

A) REST:

- Concepto y filosofía: La filosofía de esta forma esta ordenado a los recursos de forma estricta.

- Protocolo Base: Opera de forma nativa en HTTPS, aprovechando la estructura de la web.

- Operaciones y métodos: Usa principalmente los métodos GET (leer), POST (enviar), PUT (reemplazar), PATCH (modificar) y DELETE (borrar).

- Estructura de peticiones: La solicitud se envía vía URLs semánticas que exponen las rutas de recursos. El cliente realiza llamadas distribuidas a distintos puntos de forma independiente.
El payload es un JSON, en donde se ubican los métodos de escritura.

- Códigos de errores y estado: para las funciones completadas se seguirá se indicara con la saga 200, referenciando al éxito de la conexión. Para fallos del servidor o cliente sera con las sagas 500 y 400 de HTML para errores

- Pros y Contras:

- Ventajas: Cacheable por defecto a nivel de red; excelente separación de conceptos y alta familiaridad en la industria; diseño sin estado (stateless) que facilita el escalado horizontal.
- Desventajas: Propensión a la ineficiencia de datos (over-fetching y under-fetching); requiere múltiples peticiones HTTP consecutivas para construir vistas complejas; carece de un contrato estricto de tipado nativo sin herramientas externas como OpenAPI.

B) GRAPHQL:

- Concepto y filosofía: Está orientado a un grafo de datos. El modelo de dominio se concibe como un grafo interconectado, donde los nodos son entidades y las aristas son las relaciones entre ellas.

- Protocolo Base: Opera de forma nativa en HTTPS, al igual que REST, pero usando POST generalmente.

- Operaciones y métodos: Los métodos que usa son lenguajes de consulta, siendo Query, Mutation y Subcription.

- Estructura de peticiones: Se implementa a través de un punto final único. Todas las interacciones se hacen por el método POST. La carga contiene una cadena estructurada que describe los campos solicitados y la variables de entrada necesarias.

- Códigos de errores y estado: siempre se usara la saga 200 de HTTP para aciertos y errores, pero el mensaje variara entre los mismos.

- Pros y Contras:

- Ventajas: Erradica por completo el over-fetching y under-fetching al permitir al cliente pedir exactamente lo que necesita; documentación y tipado estricto auto-generado gracias al esquema del servidor; reduce el consumo de ancho de banda y el número de viajes de red.

- Desventajas: Alta complejidad para implementar mecanismos de caché tradicionales a nivel de red (ya que el punto final y el método POST no cambian); riesgo de sobrecarga del servidor ante consultas anidadas maliciosas o muy profundas; curva de aprendizaje inicial más pronunciada.

C) WEBSOCKET

- Concepto y filosofía: Su diseño es de un canal persistente, siendo su filosofía es de mantener una conexión abierta, de larga duración y bajo coste de sobrecarga.

- Protocolo Base: Se basa en protocolo con el mismo nombre, siendo independiente de HTTP, eso sí inicia usando una petición HTTP, pero después comulga con una conexión TCP distinta.

- Operaciones y métodos: No posee los métodos tradicionales para el envió de la información, el servidor y el cliente se comunican mediante código binario.

- Estructura de peticiones: No existen URLs, es un flujo continuo bidireccional de datos.

- Códigos de errores y estado: en este caso uno propio que es la saga 1000.

- Pros y Contras:

- Ventajas: Comunicación bidireccional pura en tiempo real con latencia extremadamente baja; sobrecarga de red insignificante tras el handshake inicial; ideal para el envío masivo de actualizaciones desde el servidor (server-push).

- Desventajas: Conexión con estado (stateful), lo que dificulta significativamente el escalado horizontal (requiere sincronización mediante proxies o backplanes como Redis); no es nativamente adecuado para operaciones CRUD sencillas o estáticas; vulnerable a

problemas de agotamiento de puertos en servidores si hay millones de clientes concurrentes inactivos.

3. Análisis Comparativo y Toma de Decisiones.

Eficiencia de Datos

El "over-fetching" (sobre-recuperación) ocurre cuando el servidor devuelve más datos de los necesarios para una vista. El "under-fetching" (infra-recuperación) sucede cuando un único endpoint no provee suficiente información, forzando al cliente a realizar múltiples peticiones en cadena.

GraphQL ofrece la solución más eficiente ante estos dos problemas. Debido a su lenguaje de consulta declarativo, permite especificar de forma exacta qué campos de qué recursos anidados se requieren en un solo viaje de red. El servidor procesa el grafo, une las relaciones y empaqueta la respuesta con la estructura exacta solicitada por el cliente.

Modelo de Comunicación

• Solicitud-Respuesta (REST / GraphQL): El cliente inicia la conexión enviando una petición, el servidor la procesa y devuelve una respuesta, cerrando o liberando el ciclo de comunicación. El servidor nunca puede iniciar la interacción de forma proactiva.[Cliente] ----- (Petición HTTP) ----> [Servidor]
[Cliente] <---- (Respuesta HTTP) ---- [Servidor]

• Bidireccional Persistente (WebSocket): Se abre un único canal persistente tras el apretón de manos inicial. Ambos actores, tanto el cliente como el servidor, disponen de la capacidad de inyectar datos en el canal en cualquier instante, de forma asíncrona e independiente.[Cliente] ===== (Conexión TCP Abierta) ===== [Servidor]
[Cliente] ------> (Mensaje en Evento) -----> [Servidor]
[Cliente] <------ (Mensaje Server-Push) ---- [Servidor]

Tecnologías y Ecosistemas

• REST: Implementado comúnmente con frameworks del lado del servidor como Express.js (Node.js), Spring Boot (Java) o Django Rest Framework (Python). En el navegador se consume mediante la Fetch API nativa o librerías como Axios.

• GraphQL: Implementado habitualmente mediante Apollo Server, Yoga o GraphQL.js en el backend. En el frontend se gestiona con Apollo Client o Relay.

• WebSocket: Desarrollado en el backend con la API nativa de WebSockets (módulo ws en Node.js), Socket.io o SignalR (.NET). En el lado del cliente se aprovecha el objeto nativo WebSocket provisto por las librerías del navegador (Browser API).

4. CASOS DE USO Y CONCLUSIÓN.

Justificación de Elección.

• Proyecto 1: Catálogo de Productos para un E-Commerce Público e Indexable

• Arquitectura Elegida: REST
• Justificación: Un catálogo requiere que las páginas de productos sean fácilmente indexables por los motores de búsqueda (SEO) y se beneficien de una fuerte estrategia de almacenamiento en caché. REST permite utilizar sistemas de caché perimetrales (como CDN o Varnish) basados en las URLs estáticas de los recursos. La naturaleza pública y mayormente estática de los datos hace que la semántica HTTP y la simplicidad de REST sean ideales para este escenario, garantizando un rendimiento óptimo con baja sobrecarga de cómputo en el backend.

• Proyecto 2: Panel de Gestión (Dashboard) de Analíticas Complejas con Datos Interconectados
    • Arquitectura Elegida: GraphQL
    • Justificación: Este tipo de aplicaciones móviles o web requiere renderizar vistas

heterogéneas donde se cruzan datos de múltiples entidades (métricas de ventas, perfiles de clientes, inventarios y logs de actividad). Si usáramos REST, el cliente sufriría de severo under-fetching, ralentizando el renderizado debido a decenas de llamadas a la red. GraphQL soluciona esto unificando los datos en una sola petición compacta adaptada a cada pantalla del dashboard, reduciendo el consumo de batería en móviles y simplificando el mantenimiento del código en el frontend.

• Proyecto 3: Plataforma de Negociación Financiera o Trading de Criptomonedas en Vivo
• Arquitectura Elegida: WebSocket
• Justificación: En el trading, los precios cambian múltiples veces por segundo y los retrasos de milisegundos pueden costar dinero a los usuarios. Los modelos tradicionales de solicitud-respuesta (como hacer consultas HTTP periódicas o "polling") destruyen el rendimiento del servidor y saturan la red con cabeceras repetitivas. WebSocket es la única opción viable aquí porque mantiene un canal TCP abierto donde el servidor empuja las fluctuaciones de precios instantáneamente ("server-push") con un coste de procesamiento mínimo por mensaje.

2. Conclusión Final
La investigación técnica demuestra que no existe una arquitectura superior de forma absoluta; cada una responde a necesidades operativas concretas:
• Se debe elegir REST cuando el sistema requiera robustez, simplicidad arquitectónica, un alto uso de cachés HTTP y cuando las operaciones encajen de forma natural en el paradigma CRUD de recursos independientes.
• Se debe optar por GraphQL cuando el cliente necesite flexibilidad para consumir datos altamente relacionales, reduciendo drásticamente el volumen de transferencia de red mediante la eliminación del sobre-procesamiento de payloads.
• Se debe implementar WebSocket cuando los requisitos primordiales del negocio exijan interactividad bidireccional inmediata y flujos de información en tiempo real donde la baja latencia sea una prioridad crítica.